

Incident Management UI Enhancements

Client Script & UI Policy Requirement Specification & Implementation Analysis

Project: ServiceNow Incident Workspace & Form Polish	Module / Target: Incident Management (<code>incident</code> table)
Document Phase: Phase 2 — Requirement Analysis Phase	Target Audience: Technical Leads, ServiceNow Developers, QA Engineers
Author / Role: ServiceNow Technical Consultant	Status: Baselined / Approved for Design & Build

1. EXECUTIVE SUMMARY & SCOPE

Following the initial project kickoff and stakeholder interviews in Phase 1, this document formalizes the **Requirement Analysis Phase (Phase 2)**. The primary objective is to define functional and non-functional requirements for interactive form behavior, validations, dynamic field control, and user experience enhancements on the **Incident (`incident`)** table using client-side configurations.

This phase delineates precisely when to leverage **UI Policies** (declarative, low-code/no-code, standard field manipulation) versus **Client Scripts** (procedural, programmatic event handling, complex DOM/form operations, and asynchronous GlideAjax interactions).

2. ARCHITECTURAL FRAMEWORK: UI POLICY VS. CLIENT SCRIPT

ServiceNow architectural best practices mandate declarative configuration prior to scripting. Below is the decision framework utilized during this requirement phase:

Evaluation Criterion	UI Policy Approach	Client Script Approach
Primary Use Case	Dynamically set Mandatory, Read-Only, and Visible attributes based on field condition builder.	Form submission validation, field change calculations, complex regex, cross-table GlideAjax lookups.
Execution Order	Executes <i>after</i> Client Scripts during form load.	Executes <i>before</i> UI Policies during form load.
Performance Impact	Minimal overhead; natively cached by browser runtime engine.	Can induce latency if synchronous server calls or unoptimized event handlers are used.
Maintenance Overhead	Low; manageable via system administration without coding background.	Medium-High; requires JavaScript testing, code review, and regression verification across upgrades.

3. FUNCTIONAL REQUIREMENT SPECIFICATIONS (FRS)

The requirements collected from IT Service Desk leads, Major Incident Managers, and Tier-1/Tier-2 support teams are cataloged below:

Req ID	Requirement Title	Functional Description & Conditions	Mechanism	Priority
BR-INC-01	Resolution Details Enforcement	When Incident State transitions to <code>Resolved (6)</code> or <code>Closed (7)</code> , make <code>close_code</code> and <code>close_notes</code> strictly Mandatory and Visible. If state is Active/In-Progress, keep them read-only or hidden.	UI Policy	Must Have

Req ID	Requirement Title	Functional Description & Conditions	Mechanism	Priority
BR-INC-02	P1 Incident Banner & Justification	When <code>priority == 1 (Critical)</code> , display an informational alert banner on top of the form, make <code>work_notes</code> mandatory, and show a dedicated Major Incident Candidate checkbox.	UI Policy	Must Have
BR-INC-03	Caller VIP Visual Indicator	When <code>caller_id</code> is populated or modified, asynchronously check if caller has <code>vip == true</code> . If VIP, highlight the caller field with a distinct visual badge/icon and post a prominent info message.	Client Script (onChange)	Must Have
BR-INC-04	Future Date-Time Prevention	Prevent users from setting <code>work_start</code> (Actual Work Start) or issue occurrence time into the future. Compare selected timestamp against current client/server time before saving.	Client Script (onSubmit)	Should Have
BR-INC-05	Child Incident Auto-Assignment Guard	When an incident has a populated <code>parent_incident</code> , lock Assignment Group and Assigned To fields to read-only, preventing desynchronization from the parent orchestrating ticket.	UI Policy	Should Have
BR-INC-06	Close Notes Length & Quality Check	On submit when state is <code>Resolved</code> , enforce that <code>close_notes</code> contains at least 30 characters and does not match generic placeholder phrases like "fixed", "done", or "resolved".	Client Script (onSubmit)	Should Have
BR-INC-07	Contact Type Conditionals	If <code>contact_type == 'Phone'</code> , make <code>caller_id.phone</code> visible/highlighted. If <code>contact_type == 'Walk-in'</code> , set location as mandatory.	UI Policy	Nice to Have
BR-INC-08	Auto-populate Caller Location & Dept	When <code>caller_id</code> is changed, automatically populate caller's Department, Location, and Business Phone onto the incident form without requiring full page reload.	Client Script (onChange)	Must Have

4. TECHNICAL IMPLEMENTATION SPECIFICATIONS

4.1 UI Policy Artifacts Breakdown

UIP-INC-01: Resolve/Close State Mandates

- **Table:** Incident [`incident`]
- **Condition:** `State [state]` is one of (`Resolved`, `Closed`)
- **Reverse if false:** Checked (True)
- **Global / View:** Global (All Views)
- **UI Policy Actions:**
 - `close_code` : Mandatory = True, Visible = True
 - `close_notes` : Mandatory = True, Visible = True

UIP-INC-02: Critical Priority (P1) Enforcement

- **Table:** Incident [`incident`]
- **Condition:** `Priority [priority]` is `1 - Critical`
- **Reverse if false:** Checked (True)
- **On-True Script:**

```
g_form.showFieldMsg('priority', 'CRITICAL INCIDENT: Ensure incident manager notification protocol is engaged.', 'error');
```

• **On-False Script:**

```
g_form.hideFieldMsg('priority', true);
```

4.2 Client Script Artifacts Breakdown

CS-INC-01: Caller VIP & Detail Fetch (onChange)

- **Field:** `caller_id` | **Type:** `onChange`
- **Execution Context:** Desktop & Service Portal / Agent Workspace
- **Logic:** Utilize `g_form.getReference('caller_id', callback)` or `GlideAjax` to fetch VIP flag, email, and phone. Prevent synchronous freeze.
- **UI Reaction:** If VIP is true, apply `g_form.addInfoMessage('Attention: Caller is a designated VIP executive.')`.

CS-INC-02: Resolution Verification (onSubmit)

- **Field:** N/A | **Type:** `onSubmit`
- **Trigger:** Form submission / update action.
- **Validation:** Check if `state == 6 (Resolved)`. Verify `g_form.getValue('close_notes').trim().length >= 30`.
- **Error Handling:** If validation fails, call `g_form.addErrorMessage('Close notes must be comprehensive (min 30 characters).')` and return `false`.

5. DEPENDENCIES, EDGE CASES & GOVERNANCE RULES

- **Order of Execution Conflicts:** UI Policies must not conflict with Client Scripts setting the same field attributes. If a Client Script sets a field read-only, an overlapping UI Policy reverse-if-false condition must not unintentionally overwrite it.
- **View-Rule Dependencies:** Both scripts and policies must be configured with `global = true` unless specifically scoped for the *Self-Service (ess)* or *Major Incident* view.
- **Workspace & Mobile Compatibility:** UI Type must be set to `All` (or `Mobile / Service Portal + Desktop`) to guarantee uniformity across Core UI16, Service Operations Workspace (SOW), and Mobile Agent.
- **Performance Guardrails:** Synchronous `GlideRecord ( gr.query() )` in client scripts is strictly prohibited under ServiceNow Architecture governance. `GlideAjax` with `getXMLAnswer()` must be used for any server interactions.

6. PHASE 2 DELIVERABLES & SIGN-OFF CRITERIA

Milestone Verification: This Requirement Analysis phase transitions into **Phase 3 (Design & Architecture)** upon stakeholder approval of the field mappings, triggers, and state rules outlined above.

Stakeholder Role	Representative Name	Approval Status	Sign-Off Date
ServiceNow Product Owner	Enterprise IT Service Management	Approved	Current Baseline
Incident Process Owner	Global Operations & Support	Approved	Current Baseline
Lead ServiceNow Architect	Enterprise Architecture Board	Approved	Current Baseline